

Intermediate (build real memory) — Solution

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Hint 3](#) · [Hint 4](#) · [Solution](#)

Simple Version

```
from dotenv import load_dotenv
from openai import OpenAI

load_dotenv()
client = OpenAI()

history = [
    {"role": "system", "content": "You are a helpful assistant."}
]

while True:
    user_input = input("You: ")
    if user_input == "quit":
        break

    history.append({"role": "user", "content": user_input})

    response = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=history,
    )
    reply = response.choices[0].message.content

    history.append({"role": "assistant", "content": reply})
    print("Assistant:", reply)
```

This works correctly and proves memory — try the "favorite color" test from Hint 4. It has no error handling, and it never limits how large `history` can grow, which matters once conversations get long.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Hint 3](#) · [Hint 4](#) · [Solution](#)

Intermediate Version

```
from dotenv import load_dotenv
from openai import OpenAI

load_dotenv()
client = OpenAI()

def get_reply(history: list) -> str:
    response = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=history,
    )
    return response.choices[0].message.content

def run_chat() -> None:
    history = [
        {"role": "system", "content": "You are a helpful assistant."}
    ]

    while True:
        user_input = input("You: ")
        if user_input == "quit":
            break

        history.append({"role": "user", "content": user_input})

        reply = get_reply(history)
        history.append({"role": "assistant", "content": reply})

        print("Assistant:", reply)

if __name__ == "__main__":
    run_chat()
```

What's different, and why it's better: pulling the API call out into `get_reply(history)` separates "how to talk to the model" from "how the loop is structured" — this is the exact shape

`chat_client.py`'s `send_message()` needs to take in the Build Task below. Full type hints on `get_reply` document that it takes the whole history list and returns just the new reply text, not a whole response object the caller has to unpack.

Which one should you use? The Simple version is fine to prove memory works for yourself once. The Intermediate version's separated `get_reply()` function is what belongs in real code — it's directly reusable inside the Build Task's `chat_client.py`, instead of code you'd have to restructure later.